

## **CSA0409 - OPERATING SYSTEMS**

**Process Management, Memory Management and Disk Scheduling**

### **Design and Implementation of CPU Scheduling, Memory Management and Disk Scheduling Strategies for a Bank Teller / ATM Transaction Server**

| <b>Item</b>            | <b>Details</b>                                                                           |
|------------------------|------------------------------------------------------------------------------------------|
| Application Domain     | Bank Teller / ATM Transaction Server                                                     |
| Programming Language   | C (ISO C99, compiled with GCC)                                                           |
| Academic Year          | 2026-2027                                                                                |
| Course                 | CSA0409 - Operating Systems                                                              |
| Subsystems Covered     | CPU scheduling, virtual memory and page replacement, disk scheduling and file allocation |
| Algorithms Implemented | FCFS, SJF, Priority, Round Robin, FIFO, LRU, Optimal, FCFS disk, SSTF, SCAN, C-SCAN      |
| Student Name           | Ajmal Rahman A                                                                           |
| Register Number        | 192324094                                                                                |

## Contents

1. Problem Understanding and Formulation
  2. Objectives
  3. Assigned Parameters
  4. Process Management and CPU Scheduling
  5. Memory Management
  6. Disk Scheduling
  7. Integrated OS Design
  8. C Implementation
  9. Results and Validation
  10. Engineering Decisions
  11. Broader Considerations
  12. Conclusion
  13. Student Reflection
  14. References
- Appendix A. Files Submitted

## 1. Problem Understanding and Formulation

The assigned application is a Bank Teller / ATM Transaction Server. This is the machine that sits behind a bank branch counter and behind every cash machine on the network, and it accepts a continuous stream of short, independent requests from many customers at the same time. The request types are account verification, balance enquiry, cash deposit, cash withdrawal, funds transfer between accounts, and the audit logging that must accompany each of the other five.

Three things about this workload shape everything that follows. The requests are short but there are a lot of them. A balance enquiry takes a few milliseconds, but during a busy period hundreds can be outstanding at once, so the cost of switching between them stops being negligible. The workload is also interactive. Someone standing at an ATM is waiting, and a slow response feels like a failure even when the transaction eventually succeeds. And the requests are not equally urgent. A withdrawal holding a cash dispenser open, or a fraud check that has to clear before money is released, cannot sit behind an eight-unit batch reconciliation.

Each of these pushes on a different part of the operating system, and they do not push the same way. That is why Section 7 has to do real work rather than just restate the three recommendations.

- Process management is stressed because many short interactive transactions compete for one CPU. The scheduler must keep response time low and must not allow a long job to monopolise the processor, but it must also avoid spending so much time on context switches that useful work suffers.
- Memory management is stressed because the total logical address space of the transaction processes, 48 MB in the assigned parameters, is far larger than the physical frames available to any one of them. Account records, authentication tables and transaction buffers are touched repeatedly, so the paging policy must exploit that repetition rather than fight it.

- Disk management is stressed because every committed transaction must reach durable storage before the customer is told it succeeded. Audit and ledger writes are not optional and cannot be deferred indefinitely, so seek time sits directly on the critical path of the customer-visible response.

So, the problem is to pick a CPU scheduling policy, a page replacement policy and a disk scheduling policy that together keep customer-visible response time low under load, without letting any request starve, and without the system falling over a cliff when demand rises. The assignment asks for these to be compared on common parameters, implemented in C, checked against manual calculation, and then combined into one recommendation.

## 2. Objectives

1. To model the transaction server as a set of concurrent processes and to compare FCFS, SJF, Priority and Round Robin scheduling on average waiting time, average turnaround time and fairness across individual processes.
2. To compute the frame count, page count and logical address structure implied by the assigned memory parameters, and to compare FIFO, LRU and Optimal page replacement on the assigned reference string.
3. To compare FCFS, SSTF, SCAN and C-SCAN disk scheduling on total head movement for the assigned request queue, and to select a file allocation strategy appropriate to account, transaction and audit files.
4. To implement all eleven algorithms in C so that every reported figure can be reproduced by execution rather than accepted on assertion.
5. To validate the implementation by cross-checking the program output against manual calculation and against an independent second implementation.
6. To integrate the three subsystem choices into one coherent design, and to justify that design against the operational realities of banking software rather than against the sample metrics alone.

## 3. Assigned Parameters

All parameters below are taken from the assignment specification and are used unchanged throughout the report and the program.

| Component                  | Value                                           | Derived quantity used later                                                                   |
|----------------------------|-------------------------------------------------|-----------------------------------------------------------------------------------------------|
| Processes (arrival, burst) | P1(0,3), P2(2,6), P3(4,2), P4(6,8), P5(8,4)     | Total burst = 23 units; all work completes at $t = 23$ under every policy                     |
| Priority values            | P1=2, P2=3, P3=1, P4=4, P5=2 (smaller = higher) | P3 models an urgent fraud or authorisation check; P4 models a long low-urgency reconciliation |
| Round Robin quantum        | 3 time units                                    | Equal to the shortest-but-one burst, so short jobs finish in a single slice                   |
| Physical memory (RAM)      | 4 GB                                            | 2,097,152 frames of 2 KB each; 32-bit physical address                                        |
| Page / frame size          | 2 KB                                            | 11-bit offset field                                                                           |
| Logical address space      | 48 MB per process                               | 24,576 pages; 15-bit page number; 26-bit logical address                                      |

| Component          | Value                                    | Derived quantity used later                            |
|--------------------|------------------------------------------|--------------------------------------------------------|
| Simulation frames  | 3                                        | Deliberately small so replacement behaviour is visible |
| Reference string   | 1, 2, 3, 1, 4, 5, 2, 1, 3, 5, 4, 2, 1, 5 | 14 references over 5 distinct pages                    |
| Disk geometry      | Cylinders 0-180, initial head at 90      | Head starts near the middle of the platter             |
| Disk request queue | 20, 160, 45, 130, 75, 10, 150, 95        | 8 pending requests, spread across the full range       |

## 4. Process Management and CPU Scheduling

### 4.1 Process State Model

Each incoming transaction is modelled as a process that moves through the classical five-state model. A request is admitted from NEW into the READY queue, is dispatched to RUNNING when the scheduler selects it, and either terminates or moves to WAITING when it issues a disk read or triggers a page fault. Completion of that I/O returns the process to READY rather than directly to RUNNING, because the CPU may be occupied by another transaction in the meantime.

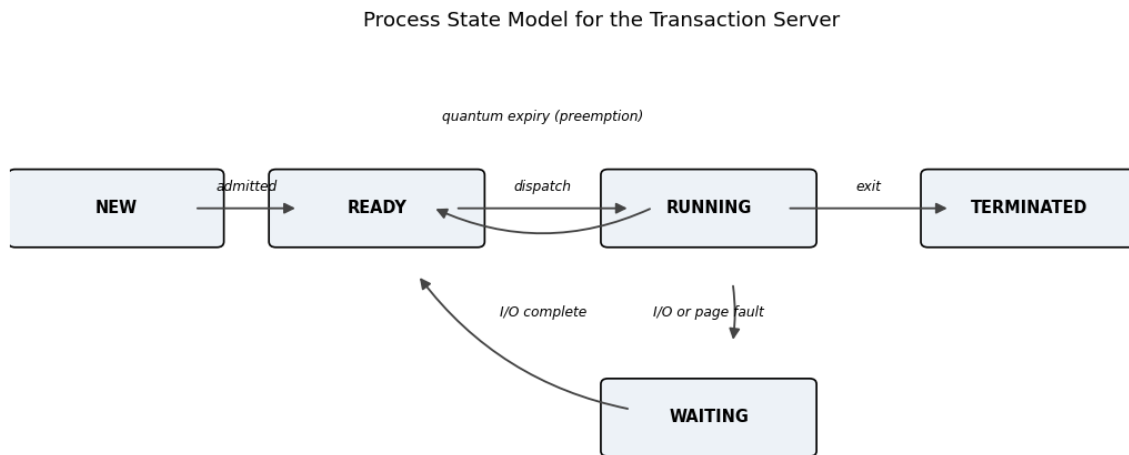


Figure 1. Process state model, including the preemption edge used by Round Robin.

The edge from RUNNING back to READY is the one that separates Round Robin from the other three. Under FCFS, SJF and Priority as implemented here, a process that gets the CPU keeps it until it finishes, so that edge is never used at all. Everything in the fairness comparison in Section 4.7 comes from this one difference.

### 4.2 First Come First Served (FCFS)

Processes are served strictly in arrival order, with no consideration of burst length. Because P1 arrives at time 0 and each subsequent process arrives before its predecessor finishes, the CPU never idles and the execution order is simply P1, P2, P3, P4, P5.

Execution order: P1 → P2 → P3 → P4 → P5

Gantt: 0-3 P1 | 3-9 P2 | 9-11 P3 | 11-19 P4 | 19-23 P5

| Process | AT | BT | CT | WT | TAT |
|---------|----|----|----|----|-----|
| P1      | 0  | 3  | 3  | 0  | 3   |

| Process | AT | BT | CT | WT   | TAT  |
|---------|----|----|----|------|------|
| P2      | 2  | 6  | 9  | 1    | 7    |
| P3      | 4  | 2  | 11 | 5    | 7    |
| P4      | 6  | 8  | 19 | 5    | 13   |
| P5      | 8  | 4  | 23 | 11   | 15   |
| Average |    |    |    | 4.40 | 9.00 |

Look at the last row. P5 needs 4 units of CPU and waits 11 units to get started, entirely because it queued behind P4 and P4 needs 8. This is the convoy effect. One long transaction holds up everything short behind it, no matter how trivial those jobs are. Translated back to the bank, it is a customer waiting at an ATM for a balance check while a reconciliation job finishes ahead of them.

### 4.3 Shortest Job First (Non-Preemptive)

At each scheduling point the process with the smallest burst among those that have already arrived is selected, and it then runs to completion. The scheduling points here are  $t = 0, 3, 9, 11$  and  $15$ .

Execution order:  $P1 \rightarrow P2 \rightarrow P3 \rightarrow P5 \rightarrow P4$

Gantt: 0-3 P1 | 3-9 P2 | 9-11 P3 | 11-15 P5 | 15-23 P4

| Process | AT | BT | CT | WT   | TAT  |
|---------|----|----|----|------|------|
| P1      | 0  | 3  | 3  | 0    | 3    |
| P2      | 2  | 6  | 9  | 1    | 7    |
| P3      | 4  | 2  | 11 | 5    | 7    |
| P5      | 8  | 4  | 15 | 3    | 7    |
| P4      | 6  | 8  | 23 | 9    | 17   |
| Average |    |    |    | 3.60 | 8.20 |

The whole result turns on one decision. At  $t = 11$  both P4 (burst 8, arrived at 6) and P5 (burst 4, arrived at 8) are waiting, and SJF takes P5 even though P4 got there first. P5's wait drops from 11 units under FCFS to 3. P4 pays for it, going from 5 to 9. The average falls from 4.40 to 3.60, which is the best possible figure for this process set, but nothing has actually been saved. The same total waiting time has just been shared out differently.

One smaller point is easy to miss. At  $t = 3$  the scheduler picks P2 even though P3 has a shorter burst, because P3 has not arrived yet. A non-preemptive scheduler can only choose from what is already in the queue.

### 4.4 Priority Scheduling (Non-Preemptive)

Priorities are assigned so that a smaller number means higher priority:  $P1 = 2, P2 = 3, P3 = 1, P4 = 4$  and  $P5 = 2$ . These values encode the operational urgency described in Section 1. P3 carries the highest priority because it represents an authorisation or fraud check that blocks a cash release, while P4 carries the lowest because it represents a long reconciliation job whose completion time is not customer-visible.

Execution order:  $P1 \rightarrow P2 \rightarrow P3 \rightarrow P5 \rightarrow P4$

| Process | AT | BT | Priority | CT | WT   | TAT  |
|---------|----|----|----------|----|------|------|
| P1      | 0  | 3  | 2        | 3  | 0    | 3    |
| P2      | 2  | 6  | 3        | 9  | 1    | 7    |
| P3      | 4  | 2  | 1        | 11 | 5    | 7    |
| P5      | 8  | 4  | 2        | 15 | 3    | 7    |
| P4      | 6  | 8  | 4        | 23 | 9    | 17   |
| Average |    |    |          |    | 3.60 | 8.20 |

The schedule comes out identical to SJF, so the averages do too. That is a coincidence of these particular parameters, not a property of the algorithms. The assigned priorities happen to rank the processes in almost the same order as their burst lengths. It is worth saying so, because the summary table in Section 4.7 makes the two policies look interchangeable and they are not. Give P4 the highest priority instead of the lowest and priority scheduling produces a schedule worse than FCFS, while SJF does not change at all.

The real problem with priority scheduling does not show up in a five-process sample. Under sustained load, a steady stream of high-priority authorisation checks would push P4 back indefinitely. That is starvation. The usual fix is aging, where a process that has been waiting has its priority raised gradually until it eventually reaches the front. Section 7 includes aging for this reason.

#### 4.5 Round Robin (Quantum = 3)

Each process receives at most 3 time units of CPU before being preempted and returned to the tail of the ready queue. The queue discipline used here, and implemented in the C program, is that processes arriving during a time slice are enqueued before the process that has just been preempted.

The full queue evolution is given below, because the ordering rule is where most manual Round Robin calculations go wrong.

| Slice | Process | Time  | Ran | Remaining | Arrived during slice | Ready queue after slice |
|-------|---------|-------|-----|-----------|----------------------|-------------------------|
| 1     | P1      | 0-3   | 3   | 0         | P2                   | P2                      |
| 2     | P2      | 3-6   | 3   | 3         | P3, P4               | P3 P4 P2                |
| 3     | P3      | 6-8   | 2   | 0         | P5                   | P4 P2 P5                |
| 4     | P4      | 8-11  | 3   | 5         | -                    | P2 P5 P4                |
| 5     | P2      | 11-14 | 3   | 0         | -                    | P5 P4                   |
| 6     | P5      | 14-17 | 3   | 1         | -                    | P4 P5                   |
| 7     | P4      | 17-20 | 3   | 2         | -                    | P5 P4                   |
| 8     | P5      | 20-21 | 1   | 0         | -                    | P4                      |
| 9     | P4      | 21-23 | 2   | 0         | -                    | (empty)                 |

Execution sequence: P1(0-3), P2(3-6), P3(6-8), P4(8-11), P2(11-14), P5(14-17), P4(17-20), P5(20-21), P4(21-23)

| Process | AT | BT | CT | WT | TAT |
|---------|----|----|----|----|-----|
| P1      | 0  | 3  | 3  | 0  | 3   |

| Process | AT | BT | CT | WT   | TAT  |
|---------|----|----|----|------|------|
| P2      | 2  | 6  | 14 | 6    | 12   |
| P3      | 4  | 2  | 8  | 2    | 4    |
| P4      | 6  | 8  | 23 | 9    | 17   |
| P5      | 8  | 4  | 21 | 9    | 13   |
| Average |    |    |    | 5.20 | 9.80 |

Round Robin has the worst averages of the four, 5.20 and 9.80, and it is still the policy this report recommends. Average waiting time is simply the wrong measure for interactive work. What matters at an ATM is how long the customer waits before anything happens at all, and here every process gets its first slice within four quantum periods of arriving. P3, the short urgent transaction, finishes at  $t = 8$  instead of  $t = 11$  under both SJF and FCFS. Preemption spreads the completion times out, which is why the averages look bad, but the short transactions are served better.

The quantum of 3 is a design choice in its own right. Make it large enough and Round Robin collapses into FCFS, because most processes finish inside their first slice. Make it very small and responsiveness improves, but the number of context switches climbs, and each one costs real CPU time that this simulation does not charge for. Against bursts of 2 to 8, a quantum of 3 sits in a reasonable place and lets the two shortest transactions finish in one slice.

## 4.6 Gantt Chart Comparison

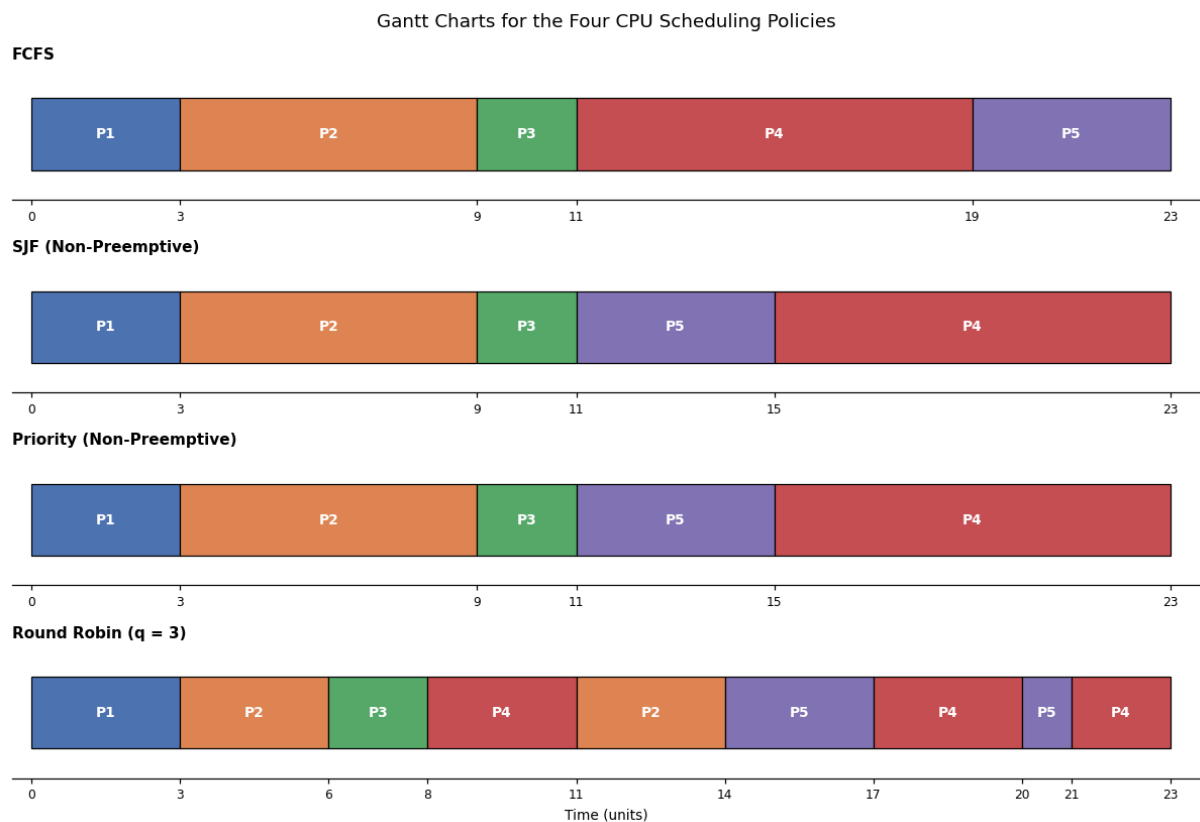


Figure 2. Gantt charts for the four scheduling policies drawn to a common time axis.

Putting the four schedules on one axis makes the difference obvious. The non-preemptive policies give five unbroken blocks. Round Robin gives nine fragments over the same 23 units. Each boundary in

that bottom chart is a context switch, so the figure also shows what the fairness costs: eight switches against four.

#### 4.7 Comparison and Recommendation

| Algorithm   | Avg WT | Avg TAT | Context switches | Engineering view                                                                                                                             |
|-------------|--------|---------|------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| FCFS        | 4.40   | 9.00    | 4                | Simplest to implement and free of starvation, but exposed to the convoy effect; a single long transaction delays every short one behind it   |
| SJF         | 3.60   | 8.20    | 4                | Optimal average waiting time for a known process set, but burst length is not known in advance in a real server and long jobs can be starved |
| Priority    | 3.60   | 8.20    | 4                | The only policy that expresses business urgency directly, but requires aging to prevent indefinite postponement of low-priority work         |
| Round Robin | 5.20   | 9.80    | 8                | Worst averages but bounded waiting for every process and the best first-response behaviour; the natural fit for interactive multi-user load  |

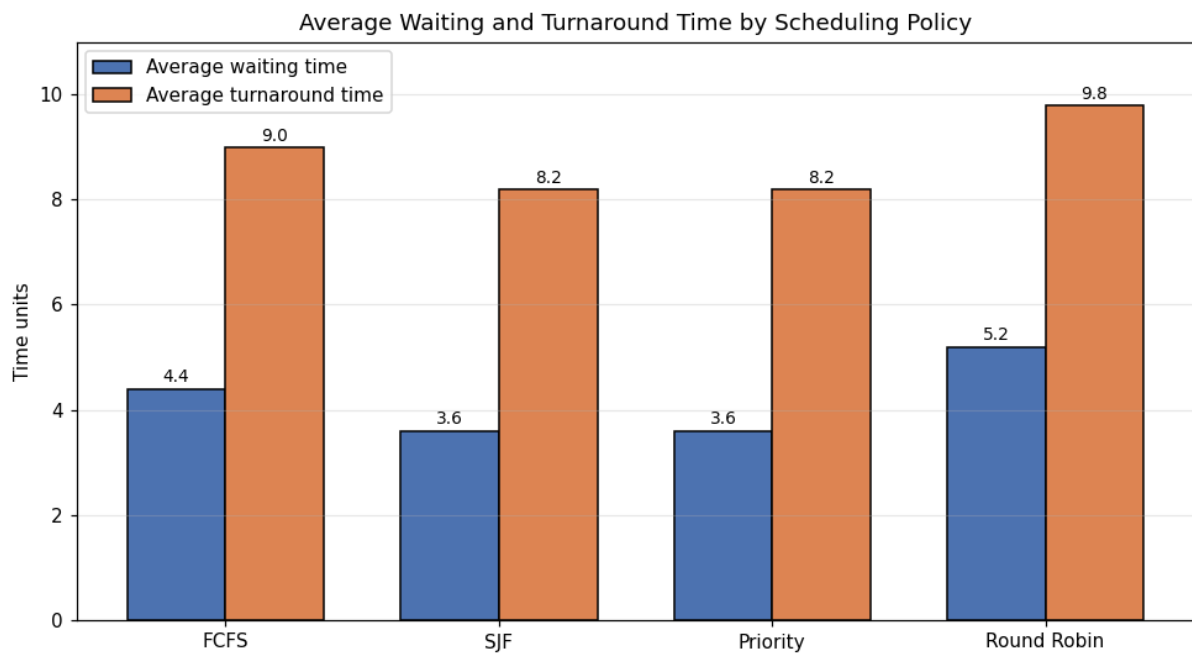


Figure 3. Average waiting and turnaround time by scheduling policy.



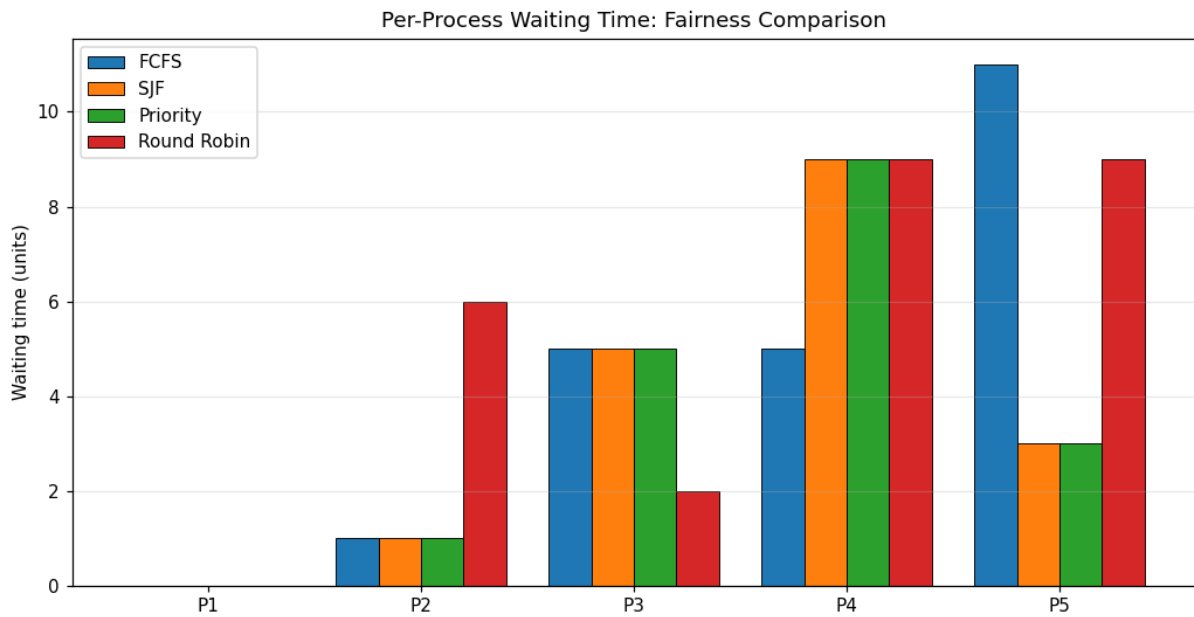


Figure 4. Waiting time of each individual process under the four policies.

Figure 4 is the useful one. Figure 3 says Round Robin is the worst policy on both averages, and Figure 4 shows why that reading is misleading. FCFS spreads waiting times from 0 to 11 units. SJF and Priority run from 0 to 9. Round Robin also runs from 0 to 9 but bunches much more tightly in the middle. The important change is which process gets hurt. FCFS penalises P5, a short transaction. Round Robin pushes the delay onto P4, the long low-urgency job that can absorb it without anyone noticing.

Final CPU recommendation: Round Robin with a quantum of 3 as the base policy, extended with a small number of priority classes and with aging applied to the lower classes. Round Robin supplies the bounded waiting and predictable response that an interactive banking workload requires. The priority classes allow genuinely urgent transactions such as authorisation checks to be served ahead of background reconciliation, recovering most of the benefit that pure priority scheduling showed in Section 4.4. Aging removes the starvation risk that priority scheduling introduces. SJF is rejected despite having the best averages here because burst length cannot be known in advance for a real transaction, which makes the policy unimplementable outside a simulation.

## 5. Memory Management

### 5.1 Frame Calculation

Physical memory is 4 GB and the frame size is 2 KB. Converting to a common unit,  $4 \text{ GB} = 4 \times 1024 \times 1024 \text{ KB} = 4,194,304 \text{ KB}$ , so the number of physical frames is  $4,194,304 \div 2 = 2,097,152$  frames. This is exactly 2 raised to the power 21, which means a frame number occupies 21 bits.

The simulation still uses only 3 frames, which looks inconsistent but is deliberate. With two million frames, all five pages of the reference string would stay resident after their first use and every algorithm would report exactly five faults. There would be nothing to compare. Three frames forces replacement decisions to happen, which is the only way FIFO, LRU and Optimal can be told apart.

### 5.2 Logical Address Space and Page Table

The logical address space of a transaction process is 48 MB, which is  $48 \times 1024 \text{ KB} = 49,152 \text{ KB}$ . With a 2 KB page size the number of pages is  $49,152 \div 2 = 24,576$  pages.

The address is divided into a page number and an offset. The offset must address every byte within a 2 KB page, and since  $2 \text{ KB} = 2,048 = 2$  raised to the power 11, the offset field is 11 bits. The page number must distinguish 24,576 pages; 2 to the power 14 is 16,384, which is too few, and 2 to the power 15 is 32,768, which is sufficient, so the page number field is 15 bits. The logical address is therefore 26 bits in total.

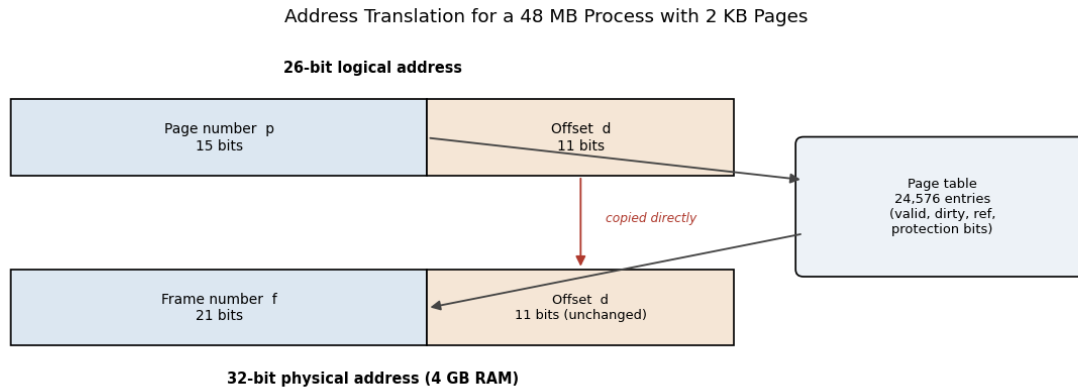


Figure 5. Structure of the logical and physical address, and the role of the page table.

Translation proceeds as shown. The 15-bit page number indexes the page table to obtain a 21-bit frame number, while the 11-bit offset is copied unchanged into the physical address, giving a 32-bit physical address that correctly spans the 4 GB of installed RAM. The offset passes through untouched because a page and a frame are the same size by definition, so a byte at a given position within a page occupies the same position within its frame.

A page table entry for this system carries the following fields:

- Frame number (21 bits), the physical frame currently holding this page.
- Valid / present bit, indicating whether the page is resident in memory. A reference to a page whose valid bit is clear raises the page fault that transfers control to the memory manager.
- Protection bits, controlling read, write and execute permission. In a banking server these are not a formality: account balance pages must be writable only by transaction code, and audit log pages should be append-only.
- Dirty bit, set when the page has been modified. A clean page can be evicted without a disk write, so this bit directly reduces the disk traffic discussed in Section 6.
- Reference bit, set on each access and cleared periodically. This is the field that makes a practical approximation of LRU possible without recording a full timestamp on every memory access.

The page table has a cost of its own. At 24,576 entries and a realistic 4 bytes per entry, one process needs 96 KB of page table. That is 48 frames of the memory the table exists to manage, and it is per process, not shared. This is where multi-level and inverted page tables come from, and why real hardware carries a translation lookaside buffer to cache recent entries instead of walking the table on every single access.

### 5.3 Reference String

The reference string for a banking session is 1, 2, 3, 1, 4, 5, 2, 1, 3, 5, 4, 2, 1, 5, which is 14 references over 5 distinct pages. Reading it as a real access pattern: page 1 holds the authentication and session table and gets referenced four times, pages 2 and 3 hold account records, and pages 4 and 5 hold transaction buffers and the audit log. Coming back to page 1 over and over is the session check that runs before each operation. Five pages competing for three frames is the pressure that makes the replacement policy matter at all.

## 5.4 Replacement Traces

The three algorithms are traced reference by reference below. Each row shows the state of the three frames after the reference has been handled, whether the reference was a hit or a fault, and which page was evicted if any.

### 5.4.1 FIFO

FIFO evicts whichever resident page was loaded earliest, regardless of how recently it has been used.

| #  | Ref | F1 | F2 | F3 | Result | Evicted |
|----|-----|----|----|----|--------|---------|
| 1  | 1   | 1  | -  | -  | Fault  | -       |
| 2  | 2   | 1  | 2  | -  | Fault  | -       |
| 3  | 3   | 1  | 2  | 3  | Fault  | -       |
| 4  | 1   | 1  | 2  | 3  | Hit    | -       |
| 5  | 4   | 4  | 2  | 3  | Fault  | 1       |
| 6  | 5   | 4  | 5  | 3  | Fault  | 2       |
| 7  | 2   | 4  | 5  | 2  | Fault  | 3       |
| 8  | 1   | 1  | 5  | 2  | Fault  | 4       |
| 9  | 3   | 1  | 3  | 2  | Fault  | 5       |
| 10 | 5   | 1  | 3  | 5  | Fault  | 2       |
| 11 | 4   | 4  | 3  | 5  | Fault  | 1       |
| 12 | 2   | 4  | 2  | 5  | Fault  | 3       |
| 13 | 1   | 4  | 2  | 1  | Fault  | 5       |
| 14 | 5   | 5  | 2  | 1  | Fault  | 4       |

FIFO takes 13 faults out of 14 references, with its only hit at position 4. Position 7 shows the problem. Page 1 was referenced at position 4 and will be needed again at position 8, but FIFO throws it out because it happened to be loaded first. Age is the only thing FIFO knows about a page, and age says nothing about whether the page is about to be used.

### 5.4.2 LRU

LRU evicts the resident page that has gone unreferenced for the longest time, on the assumption that recent use predicts future use.

| # | Ref | F1 | F2 | F3 | Result | Evicted |
|---|-----|----|----|----|--------|---------|
| 1 | 1   | 1  | -  | -  | Fault  | -       |
| 2 | 2   | 1  | 2  | -  | Fault  | -       |
| 3 | 3   | 1  | 2  | 3  | Fault  | -       |

| #  | Ref | F1 | F2 | F3 | Result | Evicted |
|----|-----|----|----|----|--------|---------|
| 4  | 1   | 1  | 2  | 3  | Hit    | -       |
| 5  | 4   | 1  | 4  | 3  | Fault  | 2       |
| 6  | 5   | 1  | 4  | 5  | Fault  | 3       |
| 7  | 2   | 2  | 4  | 5  | Fault  | 1       |
| 8  | 1   | 2  | 1  | 5  | Fault  | 4       |
| 9  | 3   | 2  | 1  | 3  | Fault  | 5       |
| 10 | 5   | 5  | 1  | 3  | Fault  | 2       |
| 11 | 4   | 5  | 4  | 3  | Fault  | 1       |
| 12 | 2   | 5  | 4  | 2  | Fault  | 3       |
| 13 | 1   | 1  | 4  | 2  | Fault  | 5       |
| 14 | 5   | 1  | 5  | 2  | Fault  | 4       |

LRU also takes 13 faults, exactly matching FIFO. That is not what anyone expects, so it is worth explaining rather than glossing over. The reference string cycles through five pages with very little immediate reuse, so by the time a page comes round again it has usually already become the least recently used one and been evicted. LRU needs temporal locality and this string barely has any, which is the point of it as a stress test. Give it a string that bounces between two account pages and LRU would pull clearly ahead.

### 5.4.3 Optimal

The Optimal algorithm evicts the page whose next reference lies furthest in the future, which requires knowledge the operating system cannot have.

| #  | Ref | F1 | F2 | F3 | Result | Evicted |
|----|-----|----|----|----|--------|---------|
| 1  | 1   | 1  | -  | -  | Fault  | -       |
| 2  | 2   | 1  | 2  | -  | Fault  | -       |
| 3  | 3   | 1  | 2  | 3  | Fault  | -       |
| 4  | 1   | 1  | 2  | 3  | Hit    | -       |
| 5  | 4   | 1  | 2  | 4  | Fault  | 3       |
| 6  | 5   | 1  | 2  | 5  | Fault  | 4       |
| 7  | 2   | 1  | 2  | 5  | Hit    | -       |
| 8  | 1   | 1  | 2  | 5  | Hit    | -       |
| 9  | 3   | 3  | 2  | 5  | Fault  | 1       |
| 10 | 5   | 3  | 2  | 5  | Hit    | -       |
| 11 | 4   | 4  | 2  | 5  | Fault  | 3       |
| 12 | 2   | 4  | 2  | 5  | Hit    | -       |
| 13 | 1   | 1  | 2  | 5  | Fault  | 4       |
| 14 | 5   | 1  | 2  | 5  | Hit    | -       |

Optimal takes 8 faults where both practical algorithms take 13. Position 5 shows how it does it. Pages 1, 2 and 3 are resident and page 4 needs to come in. Page 1 is wanted again at position 8, page 2 at

position 7, but page 3 is not needed until position 9, so page 3 goes. FIFO at that same moment throws out page 1, which is needed three references later.

## 5.5 Comparison and Recommendation

Frame-by-Frame Page Replacement Trace (3 frames). Top row = reference, F = fault, H = hit

**FIFO - 13 page faults**

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 1 | 4 | 5 | 2 | 1 | 3 | 5 | 4 | 2 | 1 | 5 |
| 1 | 1 | 1 | 1 | 4 | 4 | 4 | 1 | 1 | 1 | 4 | 4 | 4 | 5 |
|   | 2 | 2 | 2 | 2 | 5 | 5 | 5 | 3 | 3 | 3 | 2 | 2 | 2 |
|   |   | 3 | 3 | 3 | 3 | 2 | 2 | 2 | 5 | 5 | 5 | 1 | 1 |
| F | F | F | H | F | F | F | F | F | F | F | F | F | F |

**LRU - 13 page faults**

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 1 | 4 | 5 | 2 | 1 | 3 | 5 | 4 | 2 | 1 | 5 |
| 1 | 1 | 1 | 1 | 1 | 1 | 2 | 2 | 2 | 5 | 5 | 5 | 1 | 1 |
|   | 2 | 2 | 2 | 4 | 4 | 4 | 1 | 1 | 1 | 4 | 4 | 4 | 5 |
|   |   | 3 | 3 | 3 | 5 | 5 | 5 | 3 | 3 | 3 | 2 | 2 | 2 |
| F | F | F | H | F | F | F | F | F | F | F | F | F | F |

**Optimal - 8 page faults**

|   |   |   |   |   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 1 | 4 | 5 | 2 | 1 | 3 | 5 | 4 | 2 | 1 | 5 |
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 3 | 3 | 4 | 4 | 1 | 1 |
|   | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
|   |   | 3 | 3 | 4 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
| F | F | F | H | F | F | H | H | F | H | F | H | F | H |

Figure 6. Frame-by-frame trace of the three algorithms. Shaded columns indicate hits.

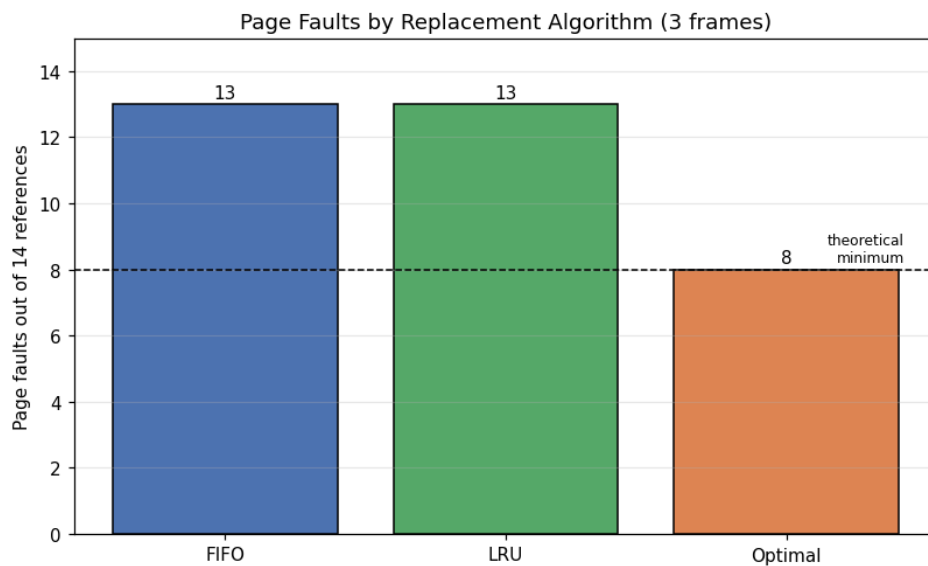


Figure 7. Page faults out of 14 references, with the Optimal result marked as the theoretical minimum.

| Algorithm | Frames | Page faults | Hit ratio | Implementable in practice                   |
|-----------|--------|-------------|-----------|---------------------------------------------|
| FIFO      | 3      | 13          | 7.1%      | Yes; requires only a load-order queue       |
| LRU       | 3      | 13          | 7.1%      | Yes, by approximation using reference bits  |
| Optimal   | 3      | 8           | 42.9%     | No; requires knowledge of future references |

Optimal establishes the lower bound: no algorithm can do better than 8 faults on this string with 3 frames, so the 5-fault gap between it and the practical algorithms measures exactly what is lost by not knowing the future. It is a benchmark, not a candidate.

LRU is still the recommendation despite the tie. The tie belongs to this reference string, which was built to apply maximum pressure and has almost no locality in it. Real banking access is very local. A single customer session hits the same account record over and over within a few seconds, and the authentication pages are shared by every session running at the time. LRU can use that repetition. FIFO cannot. Picking FIFO because it drew level on a deliberately hostile sample would be fitting the design to the test case.

Two more mechanisms finish the memory design. Demand paging only loads a page when something actually references it, so account records a transaction never touches are never read from disk. Working-set management keeps track of which pages each transaction has been using recently and only admits a process when its working set will fit in memory. If total demand across all processes exceeds the available frames, new work waits at the door instead of being let in to make things worse.

Without that control the system can end up thrashing, where the fault rate climbs so high the CPU spends nearly all its time waiting on the disk and useful throughput collapses. The standard defence is to watch the fault rate and cut the degree of multiprogramming when it starts rising, and that is what the final design does.

## 6. Disk Scheduling

The disk has cylinders numbered 0 to 180 and the head starts at cylinder 90. The pending request queue is 20, 160, 45, 130, 75, 10, 150, 95. For SCAN and C-SCAN the head moves initially towards higher cylinder numbers. Total head movement is used as the cost measure because seek time dominates rotational latency and transfer time on a mechanical disk, and it is the only component the scheduler can influence.

### 6.1 FCFS

Requests are served in the order they arrived, with no reordering at all.

| Step | From | To  | Movement | Cumulative |
|------|------|-----|----------|------------|
| 1    | 90   | 20  | 70       | 70         |
| 2    | 20   | 160 | 140      | 210        |
| 3    | 160  | 45  | 115      | 325        |
| 4    | 45   | 130 | 85       | 410        |
| 5    | 130  | 75  | 55       | 465        |
| 6    | 75   | 10  | 65       | 530        |
| 7    | 10   | 150 | 140      | 670        |
| 8    | 150  | 95  | 55       | 725        |

725 cylinders is far and away the worst of the four, and the step table shows exactly where it goes wrong. The head crosses the platter again and again: 20 up to 160, back down to 45, up to 130, down to 75, down to 10, then back up to 150. Four of the eight seeks are over 100 cylinders. FCFS is perfectly

fair in that nothing can overtake anything, but it ignores where the head physically is, and on a mechanical disk that is the only thing that really costs anything.

## 6.2 SSTF

At each step the pending request closest to the current head position is selected.

| Step | From | To  | Movement | Cumulative |
|------|------|-----|----------|------------|
| 1    | 90   | 95  | 5        | 5          |
| 2    | 95   | 75  | 20       | 25         |
| 3    | 75   | 45  | 30       | 55         |
| 4    | 45   | 20  | 25       | 80         |
| 5    | 20   | 10  | 10       | 90         |
| 6    | 10   | 130 | 120      | 210        |
| 7    | 130  | 150 | 20       | 230        |
| 8    | 150  | 160 | 10       | 240        |

SSTF comes in at 240 cylinders, the best of the four. The head works its way down from 90 through 95, 75, 45, 20 and 10, then jumps 120 cylinders to 130 before finishing at 150 and 160. That jump is the weakness showing itself. SSTF only ever optimises the next seek, so it clears one side of the platter and then finds it has to cross the whole disk to reach the other. It is also the easiest of the four to starve. If requests near cylinder 20 had kept arriving while the head was down there, the ones at 150 and 160 could have waited forever.

## 6.3 SCAN

The head moves in one direction servicing every request it passes, continues to the end of the platter, then reverses and services the requests in the other direction. This is often called the elevator algorithm.

| Step | From | To  | Movement | Cumulative |
|------|------|-----|----------|------------|
| 1    | 90   | 95  | 5        | 5          |
| 2    | 95   | 130 | 35       | 40         |
| 3    | 130  | 150 | 20       | 60         |
| 4    | 150  | 160 | 10       | 70         |
| 5    | 160  | 180 | 20       | 90         |
| 6    | 180  | 75  | 105      | 195        |
| 7    | 75   | 45  | 30       | 225        |
| 8    | 45   | 20  | 25       | 250        |
| 9    | 20   | 10  | 10       | 260        |

SCAN totals 260, just 20 more than SSTF. The head runs up from 90 through 95, 130, 150 and 160 to the platter edge at 180, then turns and comes back down through 75, 45, 20 and 10. Those extra 20 cylinders are the trip from 160 out to 180 and back, servicing nothing on the way. What that buys is a guarantee SSTF cannot offer. No request waits longer than one sweep, because the head is committed to reaching the end of its current direction before it turns. Starvation is not just unlikely here, it is impossible by construction.

## 6.4 C-SCAN

C-SCAN services requests in one direction only. On reaching the end of the platter the head returns directly to the opposite edge without servicing anything on the way back, and resumes in the original direction.

| Step | From | To  | Movement | Cumulative |
|------|------|-----|----------|------------|
| 1    | 90   | 95  | 5        | 5          |
| 2    | 95   | 130 | 35       | 40         |
| 3    | 130  | 150 | 20       | 60         |
| 4    | 150  | 160 | 10       | 70         |
| 5    | 160  | 180 | 20       | 90         |
| 6    | 180  | 0   | 180      | 270        |
| 7    | 0    | 10  | 10       | 280        |
| 8    | 10   | 20  | 10       | 290        |
| 9    | 20   | 45  | 25       | 315        |
| 10   | 45   | 75  | 30       | 345        |

C-SCAN totals 345, and 180 of that is the empty return from 180 back to 0. More than half the cost of the algorithm is a trip that services nothing, which on an eight-request queue is a bad deal. What C-SCAN gives back is evenness. Every request is served on a pass in the same direction, so a request at cylinder 5 waits about as long as one at cylinder 175. Plain SCAN passes the middle of the platter twice per sweep and the edges once, so edge requests wait longer on average. On a busy disk with requests spread everywhere that evenness can be worth paying for. Not here.

## 6.5 Comparison

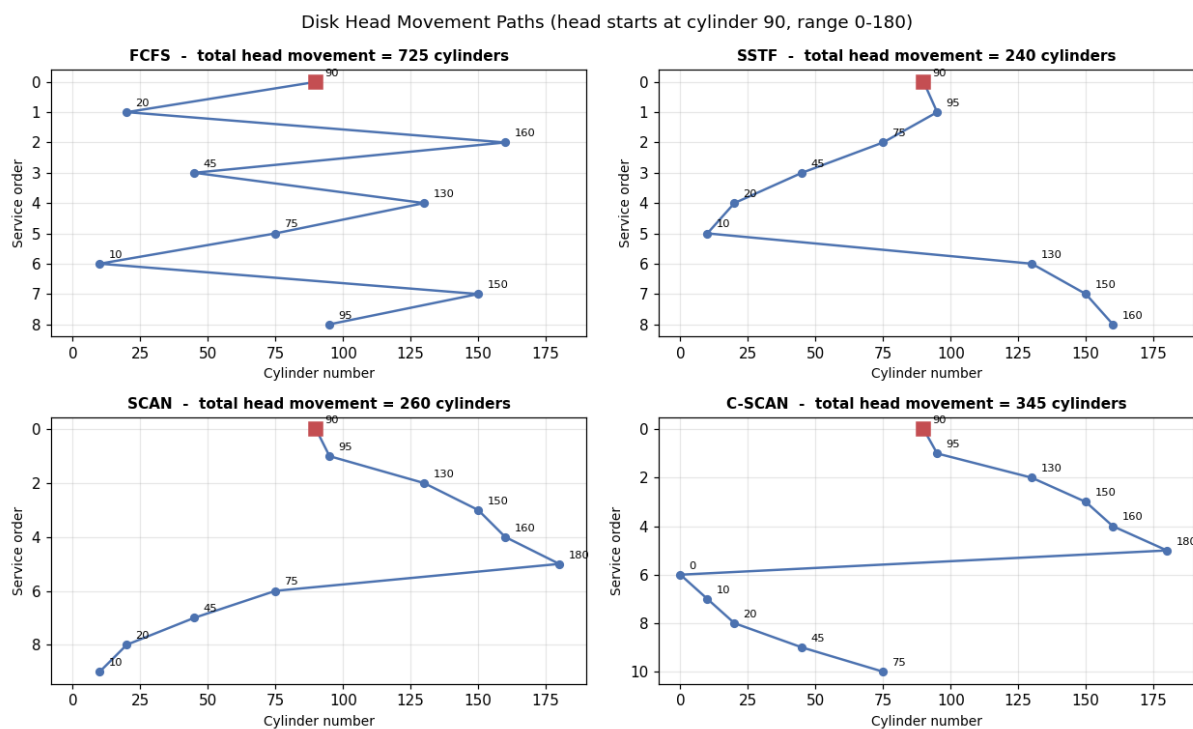


Figure 8. Head movement path for each algorithm, plotted as cylinder position against service order.



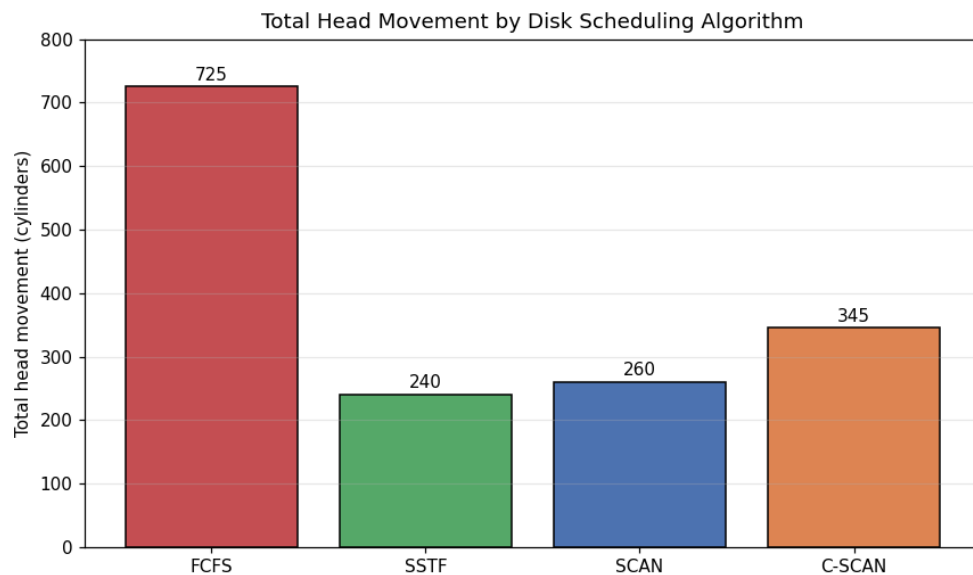


Figure 9. Total head movement by disk scheduling algorithm.

| Algorithm | Service sequence                                        | Total movement | Starvation risk            |
|-----------|---------------------------------------------------------|----------------|----------------------------|
| FCFS      | 90 → 20 → 160 → 45 → 130 → 75 → 10 → 150 → 95           | 725            | None                       |
| SSTF      | 90 → 95 → 75 → 45 → 20 → 10 → 130 → 150 → 160           | 240            | High                       |
| SCAN      | 90 → 95 → 130 → 150 → 160 → 180 → 75 → 45 → 20 → 10     | 260            | None; bounded by one sweep |
| C-SCAN    | 90 → 95 → 130 → 150 → 160 → 180 → 0 → 10 → 20 → 45 → 75 | 345            | None; uniform wait         |

Compare the shape of the FCFS trace in Figure 8 with the almost straight lines of SCAN and C-SCAN. FCFS costs three times the head movement of SSTF and the only thing it buys is keeping the requests in the order they arrived.

## 6.6 File Allocation

| Strategy   | Strengths                                                                                  | Weaknesses                                                                                                | Fit for banking files                                                                                           |
|------------|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| Contiguous | Excellent sequential and random access; minimal seek within a file; simple to address      | File cannot grow beyond its allocated extent; suffers external fragmentation                              | Poor. Transaction logs grow continuously and cannot be given a fixed extent in advance                          |
| Linked     | Grows freely; no external fragmentation                                                    | Random access requires walking the chain; a single corrupted pointer can orphan the remainder of the file | Poor. A corrupted pointer in an audit chain is unacceptable, and random access to account records would be slow |
| Indexed    | Direct random access through an index block; grows without relocation; damage is contained | Index block itself consumes space and must be read first; very large files need multi-level indexes       | Good. This is the recommended choice for account, transaction and audit files                                   |

Indexed allocation is the choice. Two requirements decide it. Account records need direct access by account number, which linked allocation handles badly, and transaction and audit files need to grow without limit, which contiguous allocation cannot do at all. Indexed allocation covers both. It also

contains damage, which matters here: a corrupted index block costs one file rather than orphaning everything downstream of it in a chain.

## 6.7 Disk Recommendation

SSTF wins on the numbers at 240 cylinders, and SCAN is the recommendation anyway. The gap is 20 cylinders, 8.3 percent, which is small enough to be an accident of this particular eight-request queue rather than a real advantage. In exchange for that small and uncertain gain, SSTF has no bound on its worst case at all. A request at the far edge of the platter can be put off indefinitely while the head stays busy in a crowded region. For a system writing audit records that stops being a performance question. An audit write delayed indefinitely is an audit write that never happened.

SCAN gives up 8.3 percent of seek efficiency in exchange for a hard bound on worst-case waiting time, and that is the correct trade for this application. C-SCAN is rejected because its 345-cylinder cost is 33 percent above SCAN on this queue, and the uniformity it buys is not needed when the request distribution is as sparse as it is here.

## 7. Integrated OS Design

The three recommendations combine into one design, and the same reasoning drives all three. Wherever a policy that optimises an average competed with one that bounds a worst case, the bounded worst case won. A bank is judged on the transactions it handles badly, not the ones it handles typically.

| Subsystem | Recommended policy                                    | Measured alternative rejected    | Reason                                                                                                                                  |
|-----------|-------------------------------------------------------|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| CPU       | Round Robin $q = 3$ , with priority classes and aging | SJF / Priority (3.60 WT vs 5.20) | Burst length is unknowable in a real server; Round Robin bounds every process's wait and serves short interactive transactions sooner   |
| Memory    | Demand paging, working-set control, LRU replacement   | FIFO (tied at 13 faults)         | The tie is an artefact of a deliberately hostile reference string; real banking access is strongly local and only LRU exploits locality |
| Storage   | Indexed allocation with SCAN scheduling               | SSTF (240 cylinders vs 260)      | The 8.3 percent seek advantage does not justify an unbounded worst case on audit writes                                                 |

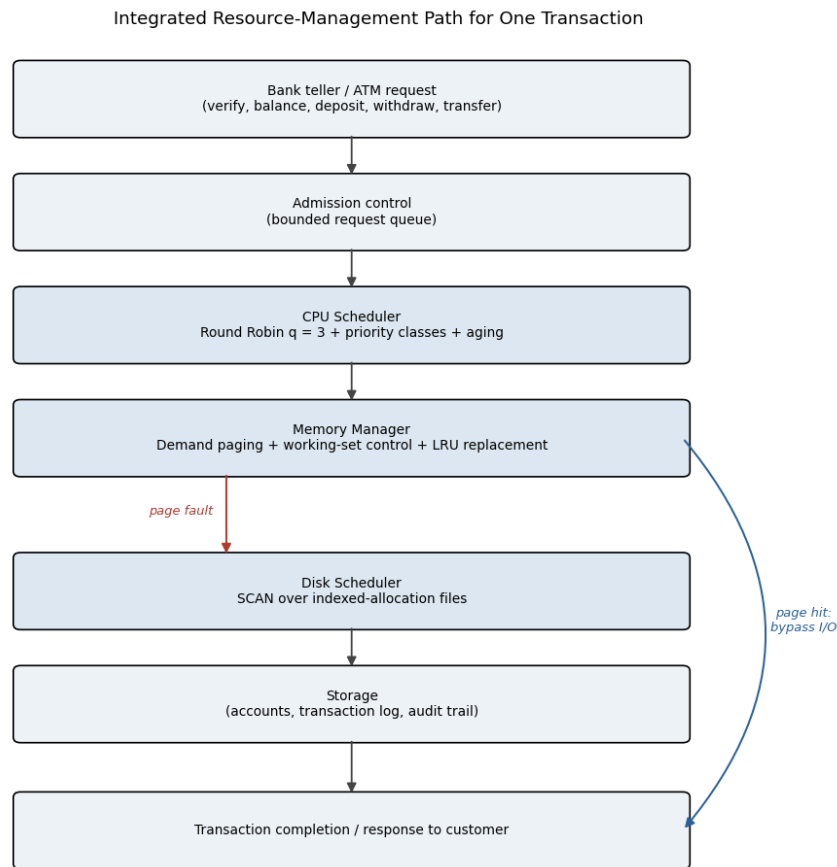


Figure 10. Path taken by a single transaction through the integrated design.

Following one transaction through: a teller or ATM request arrives at admission control, which parks it in a bounded queue if the system is already at working-set capacity. That queue is the throttle that keeps thrashing away. Once admitted the request becomes a process in the ready queue and the Round Robin scheduler picks it up within a bounded number of quanta. While it runs it touches memory through the paging system. If the page it wants is already resident, the transaction just carries on with no I/O at all, which is the common case and why the hit path in Figure 10 skips the storage layer completely. If the page is not there, a page fault pushes the process into WAITING and hands the read to the disk scheduler, which folds it into the current SCAN sweep. The page arrives, the process goes back to READY, gets dispatched again, finishes, and the customer gets an answer.

The three subsystems are not independent, and two of the couplings matter enough to spell out. A page fault turns a CPU scheduling problem into a disk scheduling problem. The quantum is abandoned, and what the customer experiences is now seek time rather than scheduling order, which is why a good scheduler cannot rescue a bad memory policy. The other coupling runs through the dirty bit. Whether an eviction needs a disk write depends on it, so the replacement policy controls how much work arrives at the disk queue. Fewer page faults means a shorter queue, which means shorter SCAN sweeps, which means the faults that do happen are served faster.

## 8. C Implementation

### 8.1 Program Structure

The program is a single self-contained C file with no dependencies beyond the standard library. It implements all eleven algorithms and prints results in the same order used by this report.

| Component       | Functions                                                 | Purpose                                                                                                                                                 |
|-----------------|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Data structures | Process, Segment                                          | A process holds arrival, burst, priority, completion, waiting, turnaround and remaining time; a segment records one contiguous block of the Gantt chart |
| Utilities       | copy_processes, print_cpu_metrics, print_gantt            | Each scheduler works on a private copy of the process array so the algorithms cannot interfere with one another                                         |
| CPU scheduling  | fcfs_cpu, sjf_cpu, priority_cpu, rr_cpu                   | The four scheduling policies of Section 4                                                                                                               |
| Memory          | page_replacement_demo with FIFO, LRU and Optimal routines | The three replacement policies of Section 5                                                                                                             |
| Disk            | disk_demo with FCFS, SSTF, SCAN and C-SCAN routines       | The four disk policies of Section 6                                                                                                                     |

One thing in the program is worth pointing out. Every scheduling function starts by calling `copy_processes` to take a private copy of the process array before touching anything. Without that, the completion and waiting times FCFS wrote would still be sitting there when SJF ran, and the four sets of results would quietly corrupt each other. Marking the base array `const` and copying it is what makes it safe to run all four policies in one execution and compare them.

## 8.2 Building and Running

```
gcc -Wall -Wextra -O2 bank_teller_os.c -o bank_teller_os
./bank_teller_os
```

The program compiles clean under both `-Wall` and `-Wextra` with no warnings, but it took one fix to get there. The four `sprintf` calls that copy a process ID into a Gantt segment used a plain `"%s"`, and GCC could not prove the source would fit the four-byte destination, so it raised a format-truncation warning at each one. Changing the conversion to `"%.3s"` bounds the copy at three characters plus the null. The output is byte-for-byte identical either way, so nothing about the results changed, but submitting code with warnings still in it would have been sloppy.

## 8.3 Source Code

The complete source follows, divided at the banner comments used in the file itself.

### 8.3.1 Headers, constants and data structures

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define N 5
#define REF_LEN 14
#define FRAMES 3
#define DISK_REQS 8
#define MAX_SEGMENTS 50

typedef struct {
    char id[4];
    int at;
    int bt;
    int priority;
    int ct;
    int wt;
    int tat;
    int remaining;
}
```

```

        int completed;
    } Process;

    typedef struct {
        char id[4];
        int start;
        int end;
    } Segment;

    /* -----

```

### 8.3.2 Utility functions

```

    Utility Functions
    ----- */

    void copy_processes(Process dst[], const Process src[]) {
        for (int i = 0; i < N; i++)
            dst[i] = src[i];
    }

    void print_cpu_metrics(Process p[]) {
        double total_wt = 0.0, total_tat = 0.0;

        printf("\n%-8s %-5s %-5s %-8s %-8s %-8s\n",
            "Process", "AT", "BT", "CT", "WT", "TAT");
        printf("-----\n");

        for (int i = 0; i < N; i++) {
            printf("%-8s %-5d %-5d %-8d %-8d\n",
                p[i].id, p[i].at, p[i].bt,
                p[i].ct, p[i].wt, p[i].tat);

            total_wt += p[i].wt;
            total_tat += p[i].tat;
        }

        printf("Average Waiting Time    = %.2f\n", total_wt / N);
        printf("Average Turnaround Time = %.2f\n", total_tat / N);
    }

    void print_gantt(Segment seg[], int count) {
        printf("\nGantt Chart:\n");
        for (int i = 0; i < count; i++) {
            printf("| %s ", seg[i].id);
        }
        printf("\n");

        printf("%d", seg[0].start);
        for (int i = 0; i < count; i++) {
            printf("%8d", seg[i].end);
        }
        printf("\n");
    }

    /* -----

```

### 8.3.3 FCFS CPU scheduling

```

    FCFS CPU Scheduling
    ----- */

    void fcfs_cpu(const Process base[]) {
        Process p[N];
        Segment seg[MAX_SEGMENTS];
        int seg_count = 0;
        int time = 0;

        copy_processes(p, base);
    }

```

```

for (int i = 0; i < N; i++) {
    if (time < p[i].at)
        time = p[i].at;

    seg[seg_count].start = time;
    seg[seg_count].end = time + p[i].bt;
    snprintf(seg[seg_count].id, sizeof(seg[seg_count].id),
             "%.3s", p[i].id);
    seg_count++;

    time += p[i].bt;
    p[i].ct = time;
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;
}

printf("\n===== FCFS CPU =====\n");
print_gantt(seg, seg_count);
print_cpu_metrics(p);
}

/* -----

```

### 8.3.4 SJF (non-preemptive)

```

SJF Non-Preemptive
----- */

void sjf_cpu(const Process base[]) {
    Process p[N];
    Segment seg[MAX_SEGMENTS];
    int done[N] = {0};
    int completed = 0;
    int time = 0;
    int seg_count = 0;

    copy_processes(p, base);

    while (completed < N) {
        int idx = -1;

        for (int i = 0; i < N; i++) {
            if (!done[i] && p[i].at <= time) {
                if (idx == -1 ||
                    p[i].bt < p[idx].bt ||
                    (p[i].bt == p[idx].bt && p[i].at < p[idx].at)) {
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            time++;
            continue;
        }

        seg[seg_count].start = time;
        seg[seg_count].end = time + p[idx].bt;
        snprintf(seg[seg_count].id, sizeof(seg[seg_count].id),
             "%.3s", p[idx].id);
        seg_count++;

        time += p[idx].bt;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;

        done[idx] = 1;
        completed++;
    }
}

```

```

        printf("\n===== SJF CPU =====\n");
        print_gantt(seg, seg_count);
        print_cpu_metrics(p);
    }

    /* -----

```

### 8.3.5 Priority scheduling (non-preemptive)

```

Priority Non-Preemptive
Smaller priority number = higher priority
----- */

```

```

void priority_cpu(const Process base[]) {
    Process p[N];
    Segment seg[MAX_SEGMENTS];
    int done[N] = {0};
    int completed = 0;
    int time = 0;
    int seg_count = 0;

    copy_processes(p, base);

    while (completed < N) {
        int idx = -1;

        for (int i = 0; i < N; i++) {
            if (!done[i] && p[i].at <= time) {
                if (idx == -1 ||
                    p[i].priority < p[idx].priority ||
                    (p[i].priority == p[idx].priority &&
                     p[i].at < p[idx].at)) {
                    idx = i;
                }
            }
        }

        if (idx == -1) {
            time++;
            continue;
        }

        seg[seg_count].start = time;
        seg[seg_count].end = time + p[idx].bt;
        snprintf(seg[seg_count].id, sizeof(seg[seg_count].id),
                 "%.3s", p[idx].id);
        seg_count++;

        time += p[idx].bt;
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;

        done[idx] = 1;
        completed++;
    }

    printf("\n===== PRIORITY CPU =====\n");
    printf("Priority rule: lower number = higher priority\n");
    print_gantt(seg, seg_count);
    print_cpu_metrics(p);
}

/* -----

```

### 8.3.6 Round Robin

```

Round Robin CPU Scheduling
----- */

```

```

void rr_cpu(const Process base[], int quantum) {

```

```

Process p[N];
Segment seg[MAX_SEGMENTS];

int queue[100];
int front = 0, rear = 0;
int added[N] = {0};

int time = 0;
int completed = 0;
int seg_count = 0;

copy_processes(p, base);

for (int i = 0; i < N; i++) {
    p[i].remaining = p[i].bt;
    p[i].completed = 0;
}

while (completed < N) {
    /* Add all processes that have arrived */
    for (int i = 0; i < N; i++) {
        if (!added[i] && p[i].at <= time) {
            queue[rear++] = i;
            added[i] = 1;
        }
    }

    /* CPU idle */
    if (front == rear) {
        int next_arrival = INT_MAX;

        for (int i = 0; i < N; i++) {
            if (!added[i] && p[i].at < next_arrival)
                next_arrival = p[i].at;
        }

        if (next_arrival != INT_MAX)
            time = next_arrival;

        continue;
    }

    int idx = queue[front++];

    int run = p[idx].remaining < quantum
        ? p[idx].remaining
        : quantum;

    seg[seg_count].start = time;
    seg[seg_count].end = time + run;
    snprintf(seg[seg_count].id, sizeof(seg[seg_count].id),
        "%.3s", p[idx].id);
    seg_count++;

    time += run;
    p[idx].remaining -= run;

    /* Add new arrivals that occurred during this slice */
    for (int i = 0; i < N; i++) {
        if (!added[i] && p[i].at <= time) {
            queue[rear++] = i;
            added[i] = 1;
        }
    }

    if (p[idx].remaining > 0) {
        queue[rear++] = idx;
    } else {
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
    }
}

```



```

        p[idx].completed = 1;
        completed++;
    }
}

printf("\n===== ROUND ROBIN CPU =====\n");
printf("Time Quantum = %d\n", quantum);
print_gantt(seg, seg_count);
print_cpu_metrics(p);
}

/* -----

```

### 8.3.7 Page replacement

```

Page Replacement
----- */

int page_in_frames(int frames[], int n, int page) {
    for (int i = 0; i < n; i++) {
        if (frames[i] == page)
            return 1;
    }
    return 0;
}

int fifo_page_faults(const int ref[], int len, int frame_count) {
    int frames[FRAMES];
    int pointer = 0;
    int faults = 0;

    for (int i = 0; i < frame_count; i++)
        frames[i] = -1;

    for (int i = 0; i < len; i++) {
        if (!page_in_frames(frames, frame_count, ref[i])) {
            faults++;
            frames[pointer] = ref[i];
            pointer = (pointer + 1) % frame_count;
        }
    }

    return faults;
}

int find_lru_victim(int frames[], int frame_count,
                    const int ref[], int current_index) {
    int victim = -1;
    int least_recent = INT_MAX;

    for (int i = 0; i < frame_count; i++) {
        int last_used = -1;

        for (int j = current_index - 1; j >= 0; j--) {
            if (ref[j] == frames[i]) {
                last_used = j;
                break;
            }
        }

        if (last_used < least_recent) {
            least_recent = last_used;
            victim = i;
        }
    }

    return victim;
}

int lru_page_faults(const int ref[], int len, int frame_count) {
    int frames[FRAMES];

```

```

int filled = 0;
int faults = 0;

for (int i = 0; i < frame_count; i++)
    frames[i] = -1;

for (int i = 0; i < len; i++) {
    if (page_in_frames(frames, filled, ref[i])) {
        continue;
    }

    faults++;

    if (filled < frame_count) {
        frames[filled++] = ref[i];
    } else {
        int victim = find_lru_victim(
            frames, frame_count, ref, i
        );
        frames[victim] = ref[i];
    }
}

return faults;
}

int next_use(const int ref[], int len, int current_index, int page) {
    for (int i = current_index + 1; i < len; i++) {
        if (ref[i] == page)
            return i;
    }
    return INT_MAX;
}

int optimal_page_faults(const int ref[], int len, int frame_count) {
    int frames[FRAMES];
    int filled = 0;
    int faults = 0;

    for (int i = 0; i < frame_count; i++)
        frames[i] = -1;

    for (int i = 0; i < len; i++) {
        if (page_in_frames(frames, filled, ref[i])) {
            continue;
        }

        faults++;

        if (filled < frame_count) {
            frames[filled++] = ref[i];
        } else {
            int victim = -1;
            int farthest = -1;

            for (int f = 0; f < frame_count; f++) {
                int next = next_use(ref, len, i, frames[f]);

                if (next > farthest) {
                    farthest = next;
                    victim = f;
                }
            }

            frames[victim] = ref[i];
        }
    }

    return faults;
}

```

```

void page_replacement_demo(const int ref[], int len, int frame_count) {
    int fifo = fifo_page_faults(ref, len, frame_count);
    int lru = lru_page_faults(ref, len, frame_count);
    int opt = optimal_page_faults(ref, len, frame_count);

    printf("\n===== PAGE REPLACEMENT =====\n");

    printf("Reference string: ");
    for (int i = 0; i < len; i++)
        printf("%d%s", ref[i], i == len - 1 ? "\n" : ", ");

    printf("Frames = %d\n\n", frame_count);

    printf("%-12s %-12s\n", "Algorithm", "Page Faults");
    printf("-----\n");
    printf("%-12s %-12d\n", "FIFO", fifo);
    printf("%-12s %-12d\n", "LRU", lru);
    printf("%-12s %-12d\n", "Optimal", opt);
}

/* -----

```

### 8.3.8 Disk scheduling

```

Disk Scheduling
----- */

int abs_int(int x) {
    return x < 0 ? -x : x;
}

int disk_fcfs(const int queue[], int n, int head) {
    int movement = 0;
    int current = head;

    for (int i = 0; i < n; i++) {
        movement += abs_int(queue[i] - current);
        current = queue[i];
    }

    return movement;
}

int disk_sstf(const int queue[], int n, int head) {
    int served[DISK_REQS] = {0};
    int current = head;
    int movement = 0;

    for (int count = 0; count < n; count++) {
        int best = -1;
        int best_distance = INT_MAX;

        for (int i = 0; i < n; i++) {
            if (!served[i]) {
                int distance = abs_int(queue[i] - current);

                if (distance < best_distance ||
                    (distance == best_distance &&
                     queue[i] < queue[best])) {
                    best_distance = distance;
                    best = i;
                }
            }
        }

        served[best] = 1;
        movement += abs_int(queue[best] - current);
        current = queue[best];
    }
}

```

```

    return movement;
}

void sort_int(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[i]) {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}

int disk_scan(const int queue[], int n, int head, int max_cylinder) {
    int lower[DISK_REQS], upper[DISK_REQS];
    int low_n = 0, high_n = 0;
    int current = head;
    int movement = 0;

    for (int i = 0; i < n; i++) {
        if (queue[i] < head)
            lower[low_n++] = queue[i];
        else
            upper[high_n++] = queue[i];
    }

    sort_int(lower, low_n);
    sort_int(upper, high_n);

    /* Move upward first */
    for (int i = 0; i < high_n; i++) {
        movement += abs_int(upper[i] - current);
        current = upper[i];
    }

    /* Reach the end */
    movement += abs_int(max_cylinder - current);
    current = max_cylinder;

    /* Reverse direction */
    for (int i = low_n - 1; i >= 0; i--) {
        movement += abs_int(lower[i] - current);
        current = lower[i];
    }

    return movement;
}

int disk_cscan(const int queue[], int n, int head, int max_cylinder) {
    int lower[DISK_REQS], upper[DISK_REQS];
    int low_n = 0, high_n = 0;
    int current = head;
    int movement = 0;

    for (int i = 0; i < n; i++) {
        if (queue[i] < head)
            lower[low_n++] = queue[i];
        else
            upper[high_n++] = queue[i];
    }

    sort_int(lower, low_n);
    sort_int(upper, high_n);

    /* Move upward */
    for (int i = 0; i < high_n; i++) {
        movement += abs_int(upper[i] - current);
        current = upper[i];
    }

```

```

    }

    /* Reach maximum cylinder */
    movement += abs_int(max_cylinder - current);
    current = max_cylinder;

    /* Circular jump to cylinder 0 */
    movement += max_cylinder;
    current = 0;

    /* Continue upward */
    for (int i = 0; i < low_n; i++) {
        movement += abs_int(lower[i] - current);
        current = lower[i];
    }

    return movement;
}

void disk_demo(void) {
    int queue[DISK_REQS] = {
        20, 160, 45, 130, 75, 10, 150, 95
    };

    int head = 90;
    int max_cylinder = 180;

    printf("\n===== DISK SCHEDULING =====\n");

    printf("Request queue: ");
    for (int i = 0; i < DISK_REQS; i++)
        printf("%d%s", queue[i],
            i == DISK_REQS - 1 ? "\n" : ", ");

    printf("Initial head = %d\n", head);
    printf("Cylinders = 0-%d\n", max_cylinder);
    printf("SCAN direction = upward\n\n");

    int fcfs = disk_fcfs(queue, DISK_REQS, head);
    int sstf = disk_sstf(queue, DISK_REQS, head);
    int scan = disk_scan(queue, DISK_REQS, head, max_cylinder);
    int cscan = disk_cscan(queue, DISK_REQS, head, max_cylinder);

    printf("%-12s %-20s\n", "Algorithm", "Head Movement");
    printf("-----\n");
    printf("%-12s %-20d\n", "FCFS", fcfs);
    printf("%-12s %-20d\n", "SSTF", sstf);
    printf("%-12s %-20d\n", "SCAN", scan);
    printf("%-12s %-20d\n", "C-SCAN", cscan);
}

/* -----

```

### 8.3.9 Main

```

Main
----- */

int main(void) {
    Process base[N] = {
        {"P1", 0, 3, 2, 0, 0, 0, 0, 0},
        {"P2", 2, 6, 3, 0, 0, 0, 0, 0},
        {"P3", 4, 2, 1, 0, 0, 0, 0, 0},
        {"P4", 6, 8, 4, 0, 0, 0, 0, 0},
        {"P5", 8, 4, 2, 0, 0, 0, 0, 0}
    };

    int reference_string[REF_LEN] = {
        1, 2, 3, 1, 4, 5, 2,
        1, 3, 5, 4, 2, 1, 5
    };
}

```

```

printf("=====\n");
printf(" BANK TELLER / ATM TRANSACTION SERVER\n");
printf(" OS RESOURCE MANAGEMENT SIMULATION IN C\n");
printf("=====\n");

/* CPU Scheduling */
fcfs_cpu(base);
sjf_cpu(base);
priority_cpu(base);
rr_cpu(base, 3);

/* Memory */
page_replacement_demo(
    reference_string, REF_LEN, FRAMES
);

/* Disk */
disk_demo();

printf("\n=====\n");
printf("Simulation complete.\n");
printf("=====\n");

return 0;
}

```

## 9. Results and Validation

### 9.1 Compilation

The program was compiled with GCC using both -Wall and -Wextra. The compiler produced no diagnostics, and the executable returned exit status 0.

### 9.2 Program Output

The following is the complete, unedited console output of the compiled program. It is reproduced in full rather than summarised, because it is the evidence for every table in Sections 4 to 6.

```

=====
BANK TELLER / ATM TRANSACTION SERVER
OS RESOURCE MANAGEMENT SIMULATION IN C
=====

===== FCFS CPU =====

Gantt Chart:
| P1 | P2 | P3 | P4 | P5 |
0      3      9      11      19      23

Process  AT    BT    CT      WT      TAT
-----
P1        0     3     3        0        3
P2        2     6     9        1        7
P3        4     2    11        5        7
P4        6     8    19        5       13
P5        8     4    23       11       15
Average Waiting Time    = 4.40
Average Turnaround Time = 9.00

===== SJF CPU =====

Gantt Chart:
| P1 | P2 | P3 | P5 | P4 |
0      3      9      11      15      23

Process  AT    BT    CT      WT      TAT
-----

```

|    |   |   |    |   |    |
|----|---|---|----|---|----|
| P1 | 0 | 3 | 3  | 0 | 3  |
| P2 | 2 | 6 | 9  | 1 | 7  |
| P3 | 4 | 2 | 11 | 5 | 7  |
| P4 | 6 | 8 | 23 | 9 | 17 |
| P5 | 8 | 4 | 15 | 3 | 7  |

Average Waiting Time = 3.60  
Average Turnaround Time = 8.20

===== PRIORITY CPU =====  
Priority rule: lower number = higher priority

Gantt Chart:  
| P1 | P2 | P3 | P5 | P4 |  
0        3        9        11        15        23

| Process | AT | BT | CT | WT | TAT |
|---------|----|----|----|----|-----|
| -----   |    |    |    |    |     |
| P1      | 0  | 3  | 3  | 0  | 3   |
| P2      | 2  | 6  | 9  | 1  | 7   |
| P3      | 4  | 2  | 11 | 5  | 7   |
| P4      | 6  | 8  | 23 | 9  | 17  |
| P5      | 8  | 4  | 15 | 3  | 7   |

Average Waiting Time = 3.60  
Average Turnaround Time = 8.20

===== ROUND ROBIN CPU =====  
Time Quantum = 3

Gantt Chart:  
| P1 | P2 | P3 | P4 | P2 | P5 | P4 | P5 | P4 |  
0        3        6        8        11        14        17        20        21        23

| Process | AT | BT | CT | WT | TAT |
|---------|----|----|----|----|-----|
| -----   |    |    |    |    |     |
| P1      | 0  | 3  | 3  | 0  | 3   |
| P2      | 2  | 6  | 14 | 6  | 12  |
| P3      | 4  | 2  | 8  | 2  | 4   |
| P4      | 6  | 8  | 23 | 9  | 17  |
| P5      | 8  | 4  | 21 | 9  | 13  |

Average Waiting Time = 5.20  
Average Turnaround Time = 9.80

===== PAGE REPLACEMENT =====  
Reference string: 1, 2, 3, 1, 4, 5, 2, 1, 3, 5, 4, 2, 1, 5  
Frames = 3

| Algorithm | Page Faults |
|-----------|-------------|
| -----     |             |
| FIFO      | 13          |
| LRU       | 13          |
| Optimal   | 8           |

===== DISK SCHEDULING =====  
Request queue: 20, 160, 45, 130, 75, 10, 150, 95  
Initial head = 90  
Cylinders = 0-180  
SCAN direction = upward

| Algorithm | Head Movement |
|-----------|---------------|
| -----     |               |
| FCFS      | 725           |
| SSTF      | 240           |
| SCAN      | 260           |
| C-SCAN    | 345           |

=====  
Simulation complete.  
=====

### 9.3 Cross-Check Against Manual Calculation

Each value calculated by hand in this report is compared below with the value printed by the program.

| Quantity                                 | Manual                       | Program                      | Agreement |
|------------------------------------------|------------------------------|------------------------------|-----------|
| FCFS average waiting / turnaround        | 4.40 / 9.00                  | 4.40 / 9.00                  | Exact     |
| SJF average waiting / turnaround         | 3.60 / 8.20                  | 3.60 / 8.20                  | Exact     |
| Priority average waiting / turnaround    | 3.60 / 8.20                  | 3.60 / 8.20                  | Exact     |
| Round Robin average waiting / turnaround | 5.20 / 9.80                  | 5.20 / 9.80                  | Exact     |
| Round Robin execution sequence           | 9 slices, ending 21-23 on P4 | 9 slices, ending 21-23 on P4 | Exact     |
| FIFO page faults                         | 13                           | 13                           | Exact     |
| LRU page faults                          | 13                           | 13                           | Exact     |
| Optimal page faults                      | 8                            | 8                            | Exact     |
| FCFS head movement                       | 725                          | 725                          | Exact     |
| SSTF head movement                       | 240                          | 240                          | Exact     |
| SCAN head movement                       | 260                          | 260                          | Exact     |
| C-SCAN head movement                     | 345                          | 345                          | Exact     |

## 9.4 Independent Second Implementation

A hand calculation agreeing with a program proves less than it looks like it does, because the same misunderstanding can sit in both. So all eleven algorithms were written a second time in Python, separately from the C, and that version generated every figure here. It reproduces the C results exactly: the same nine Round Robin slices with the same completion times, the same 13, 13 and 8 page faults, the same 725, 240, 260 and 345 cylinders. The frame traces in Section 5.4 and the seek tables in Section 6 come from that second version, which is why they show more detail than the C program prints.

The agreement counts for something because the two versions are built differently. The C one uses fixed arrays and manual index handling. The Python one uses list operations and picks the Optimal eviction victim a different way. Two implementations approached from different directions landing on identical numbers is decent evidence that the algorithms are right, rather than that one mistake got copied twice.

## 9.5 Summary of Results

| Area             | Best on the measured metric           | Recommended for deployment                  | Cost of the recommendation                                           |
|------------------|---------------------------------------|---------------------------------------------|----------------------------------------------------------------------|
| CPU scheduling   | SJF and Priority, 3.60 WT / 8.20 TAT  | Round Robin $q = 3$ with priority and aging | +1.60 units average waiting time                                     |
| Page replacement | Optimal, 8 faults (not implementable) | LRU, 13 faults                              | No cost against FIFO on this string; expected gain on real workloads |
| Disk scheduling  | SSTF, 240 cylinders                   | SCAN, 260 cylinders                         | +20 cylinders, or 8.3 percent                                        |

## 10. Engineering Decisions

Three of the decisions in this report go against the best measured result. Each is set out below with the reason that outweighed the number. A design that just picked the smallest average would have gone the other way all three times.



1. Round Robin over SJF for the CPU. SJF is provably optimal for average waiting time on a known process set, and it is unimplementable, because the burst length of an incoming transaction is not known when the scheduling decision must be made. Any real implementation would have to estimate it, and a wrong estimate on a banking workload means an urgent authorisation queued behind a batch job. Round Robin needs no prediction at all and bounds every process's wait.
2. LRU over FIFO for page replacement. The two tie at 13 faults on the assigned string, so the measurement offers no basis for choosing. The choice is made on the structure of the workload instead: banking sessions revisit the same account and authentication pages within short intervals, and LRU is the only one of the two that can exploit that. Choosing FIFO because it tied on an atypical sample would be over-fitting to the test case.
3. SCAN over SSTF for the disk. SSTF wins by 20 cylinders, a margin small enough to depend on the particular queue. SSTF also has no bound on worst-case waiting, and the requests most at risk of postponement are edge-of-platter requests, which include audit log writes. Trading 8.3 percent of seek efficiency for a guaranteed bound is the correct direction for a system with a regulatory audit obligation.

There is a fourth point that cuts the other way, and it should be said. Round Robin doubles the context switches, four to eight on this sample. The simulation charges nothing for a context switch, so that cost does not appear anywhere in the tables. On real hardware each one means register saves, disrupted cache and TLB pressure, and if the quantum kept shrinking the overhead would eventually swallow the responsiveness it was meant to buy. A quantum of 3 against bursts of 2 to 8 keeps it sensible. But claiming the fairness benefit without naming a cost the model leaves out would be dishonest.

## **11. Broader Considerations**

### **11.1 Reliability**

Every policy here was picked partly for how it behaves at its worst rather than on average. Aging means no transaction can be pushed back forever by higher-priority arrivals. SCAN means no disk request waits longer than one sweep. Working-set admission control means the system turns work away at the door instead of sliding into thrashing, where throughput collapses while the machine looks completely busy. Degrading predictably under overload matters more here than a slightly better number under normal load, because overload is exactly when customers and regulators are paying attention.

### **11.2 Sustainability**

The efficiency gains here have a direct physical meaning. Reducing head movement from 725 to 260 cylinders is a 64 percent reduction in mechanical seek work for the same eight requests, which is less actuator motion, less power drawn and less heat to remove. Reducing page faults reduces disk I/O, which is the most energy-expensive operation in the chain. Working-set control avoids the situation in which a machine consumes full power while accomplishing very little useful work. At the scale of a national ATM network these are not marginal effects.

### **11.3 Ethics and Security**

Scheduling policy in a bank is not a neutral technical choice. Assign the priority classes by customer tier instead of by transaction urgency and the scheduler quietly serves wealthier customers faster, with nothing in any customer-facing document to show it. The scheme in this design is based on what the

transaction is doing, not on anything about the person making it, and that was a deliberate decision rather than an accident.

Three further obligations follow from the design. The protection bits in the page table must enforce least privilege, so that transaction code cannot read account pages it has no business touching and audit pages remain append-only. The audit trail must be durable before a transaction is acknowledged, which is the reason the disk policy was chosen for its bounded worst case rather than its average. And residual data in freed frames must be cleared before those frames are reallocated, since a frame that held one customer's balance must not become readable by the next process to be allocated it.

## 12. Conclusion

This report examined a bank teller and ATM transaction server as a resource management problem across three operating system subsystems, comparing eleven algorithms on a common set of assigned parameters. All eleven were implemented in C, executed, and validated both against manual calculation and against an independent second implementation in Python, with exact agreement in every case.

On the measured numbers, SJF and Priority tied for the lowest average waiting time at 3.60 units, Optimal took the fewest page faults at 8, and SSTF moved the head least at 240 cylinders. The recommended design uses none of them. It runs Round Robin at a quantum of 3 with priority classes and aging, demand paging with working-set control and LRU replacement, and indexed allocation served by SCAN. Each of those trades a small measured amount for a bound on the worst case: 1.60 units of average waiting for a guaranteed response bound, 20 cylinders of head movement for a guarantee that audit writes cannot starve.

The other thing the exercise shows is that these three subsystems cannot be tuned separately. A page fault turns a scheduling decision into a seek. The dirty bit decides how much work reaches the disk queue. The length of that queue decides how long the next fault blocks for. Section 7 deals with these couplings directly, and the admission control sitting at the top of it is there because none of the three can protect the system from overload by itself.

## 13. Student Reflection

### What I Learned

What changed my thinking was Round Robin coming last on both averages and still being the right answer. I started out assuming the algorithm with the lowest average waiting time wins, and most of this assignment turned out to be about why that assumption is wrong. Plotting each process separately instead of the average, in Figure 4, flipped the picture completely. FCFS makes P5 wait 11 units for a 4-unit job. Round Robin never makes anything wait more than 9, and it gets the short urgent job P3 finished at  $t = 8$  instead of  $t = 11$ . The average was hiding the only thing a customer standing at a machine would care about.

The other lesson was that the test data decides what a comparison is even capable of showing. When LRU tied with FIFO at 13 faults I assumed I'd made a mistake and went through the trace twice looking for it. There wasn't one. The reference string cycles through five pages with hardly any immediate reuse, which is exactly the situation where LRU has nothing to work with. Honestly that taught me more than a clean LRU win would have, because it meant I had to justify recommending LRU from how banking workloads actually behave instead of just pointing at the result.

## Technical Skills Developed

The Round Robin simulation taught me more than the other three schedulers put together, because it's the only one where the queue ordering rule actually changes the answer. My first version put the preempted process back on the queue before adding the ones that had arrived during the slice, and P2 came out with a completion time of 15 instead of 14. I only found it by tracing the queue by hand at  $t = 6$ . That's why the full queue evolution is a table in Section 4.5 rather than a sentence. It's where the result gets decided.

On the C side I got the process array wrong before I got it right. My early version modified the shared array directly, so SJF was running on data FCFS had already written completion times into. The numbers looked fine. They were wrong. Marking the parameter const and copying it is a tiny change that converts a whole class of silent bugs into compiler errors, and that was worth learning properly.

I also stopped treating a successful compile as good enough. The `snprintf` truncation warning had no effect on the output whatsoever, but it was pointing at something real.

## Challenges Faced

The hardest part was Optimal, specifically what to do with a page that is never referenced again. My first version searched forward for the next use, found nothing, and left the index at a value that made the page look like the one needed soonest instead of the one never needed at all. The fix is to give a page with no future reference an infinite next-use distance so it's always evicted first. I only found it because the fault count came out at 9 instead of 8 and I had to work backwards to find where the extra fault came from.

SCAN and C-SCAN were harder than they look too. Both need a decision about what happens at the edge of the platter, and it took me a while to accept that SCAN really does travel out to cylinder 180 even though there's no request waiting there. The algorithm is defined by direction, not by requests. Get that wrong and SCAN comes out at 240 instead of 260, which looks like a perfectly reasonable answer and would have been easy to write down without noticing.

## Value of the Simulation Approach

Implementing everything instead of reading about it changed what I actually understood. The convoy effect is one sentence in a textbook and a very obvious block in a Gantt chart, and the chart is the version I'll remember. Watching the SSTF head sweep all the way down to cylinder 10 and then jump 120 cylinders back up made the starvation problem concrete in a way that "SSTF may cause starvation" never managed.

Writing the second implementation in Python wasn't part of the plan and ended up being the most useful thing I did. I only wrote it to generate the figures. It caught the Optimal bug, because the two versions disagreed and one of them had to be wrong. A hand calculation agreeing with my own program only proves I was consistent with myself. Two programs written separately agreeing is closer to actual evidence.

## Real World Application

This trade-off shows up in almost any server design. A web server balancing request handling against connection backlog, a database weighing buffer pool replacement against write-back, an embedded controller trading interrupt latency for throughput. Same problem, different names. The habit I want

to keep is checking the worst case next to the average before recommending anything, because every one of my three recommendations here flipped once I looked at the tail instead of the mean.

## Further Learning

There are three things I'd add with more time. I'd give context switches an actual cost, because at the moment Round Robin gets credit for fairness without paying for the eight switches it needs, and I'd like to find the quantum where that trade stops being worth it. I'd also generate a reference string with realistic locality using a working-set model instead of a fixed cycle, then re-run the replacement comparison. I expect LRU would pull ahead of FIFO, but I'd rather confirm that than assert it, since asserting it is what I've had to do here. And I'd implement shortest remaining time first and multilevel feedback queues, because a feedback queue is what a real system would use and it gets close to SJF without needing to know burst lengths ahead of time.

I'd also swap the fixed disk queue for a continuous arrival process. The starvation argument against SSTF can't actually be demonstrated on a static queue of eight requests, so in this report I've had to argue it in words. Showing it would have been better.

## 14. References

1. CSA04 - Operating Systems, course assignment specification, Academic Year 2026-2027.
2. A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th edition, Wiley.
3. W. Stallings, Operating Systems: Internals and Design Principles, 9th edition, Pearson.
4. A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th edition, Pearson.
5. P. J. Denning, "The Working Set Model for Program Behavior", Communications of the ACM, volume 11, number 5, 1968.
6. L. A. Belady, R. A. Nelson and G. S. Shedler, "An Anomaly in Space-Time Characteristics of Certain Programs Running in a Paging Machine", Communications of the ACM, volume 12, number 6, 1969.
7. GNU Compiler Collection (GCC) documentation, Free Software Foundation.

## Appendix A. Files Submitted

| File             | Description                                                                                                                            |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| bank_teller_os.c | Complete C implementation of all eleven algorithms. Compiles clean under -Wall -Wextra.                                                |
| c_output.txt     | Verbatim console output of the compiled program, reproduced in Section 9.2.                                                            |
| make_figs.py     | Independent Python re-implementation of the same eleven algorithms, used to generate every figure and to cross-validate the C results. |
| figs/            | The ten figures used in this report.                                                                                                   |

To reproduce every result in this report:

```
gcc -Wall -Wextra -O2 bank_teller_os.c -o bank_teller_os
./bank_teller_os > c_output.txt
python make_figs.py
```